

DAYANANDA SAGAR COLLEGE OF ENGINEERING

(An Autonomous Institution Affiliated to VTU, Belagavi)

SHAVIGE MALLESHWARA HILLS, K.S.LAYOUT, BANGALORE-560078

Department of Computer Science and Engineering



2023-2024

IV Semester

Design and Analysis of Algorithms

Laboratory Manual

Compiled by

Prof. A M Prasad

Prof. Aruna S

Prof. Nidhishree M S

DAYANANDA SAGAR COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Vision and Mission of the Department

Vision Statement

To provide a vibrant learning environment in Computer Science and Engineering with focus on industry needs and research, for the students to be successful global professionals contributing to the society.

Mission Statement

- To adopt a contemporary teaching learning process with emphasis on hands on and collaborative learning
- To facilitate skill development through additional training and encourage student forums for enhanced learning.
- To collaborate with Industry partners and Professional Societies and make the students industry ready.
- To encourage innovation through multidisciplinary research and development activities
- To inculcate human values and ethics in students and groom them to be responsible citizens.

Part – B

These are the problems from regular syllabus. Students are required to

1. Understand problem.
2. Develop mathematical Solution.
3. Algorithm.
4. Program.

Support will be given only till algorithm development. The student required to convert algorithmic solution to the programmable solution. These problems are considered for laboratory semester end examination.

#	Problem statement	Hours	COs
1	Develop the logic using <u>Quick sort</u> strategy and implement the program to sort a given set of elements and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph for the time taken versus n. The elements should be read from a file/can be generated using the random number generator.	02	
2	Develop the logic using <u>Merge sort</u> strategy and implement the program to sort a given set of elements and determine the time required to sort the elements. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph for the time taken versus n. The elements should be read from a file/can be generated using the random number generator.	02	
3	Write and execute a menu driven program to print all the nodes reachable from a given starting node in a graph using DFS and BFS method.	02	
4	Write and execute a menu driven program to print all the nodes reachable from a given starting node in a graph to arrange nodes in topological order using DFS method and source removal technique.	02	
5	Develop the logic using Horspool String Matching algorithm to implement a program to search for the given pattern in given text string.	02	
6	Write and execute a program for matrix chain multiplication for the given sequence of Matrices { A ₁ , A ₂ A _n }.	02	
7	Write and execute a program to find a longest-common-subsequence of X and Y using dynamic programming for given two sequences X = {x ₁ , x ₂ x _m }; Y = {y ₁ y ₂ y _n }	02	
8	Write and execute a program to find Minimum Spanning Tree using Prim's method.	02	
9	Write and execute a program to find Minimum Spanning Tree using Kruskal's method.	02	
11	Write and execute a program to find shortest path to all other nodes in weighted graph using Dijkstra's strategy.	02	

10	Write and execute a program to find a subset of a given set $S = \{s_1, s_2, \dots, s_n\}$ of n positive integers whose sum is equal to a given positive integer d . For example, if $S = \{1, 2, 5, 6, 8\}$ and $d = 9$ there are two solutions $\{1, 2, 6\}$ and $\{1, 8\}$. A suitable message is to be displayed if the given problem instance doesn't have a solution	02	
12	Write and execute a program to find solution to n -queens problem	02	

- Develop the logic using Quick sort strategy and implement the program to sort a given set of elements and determine the time required to sort the elements. Repeat the experiment for different values of n , the number of elements in the list to be sorted and plot a graph for the time taken versus n . The elements should be read from a file/can be generated using the random number generator.**

//Sorts a subarray by quicksort

//Input: Subarray of array $A[0..n - 1]$, defined by its left and right indices l and r

//Output: Subarray $A[l..r]$ sorted in nondecreasing order

if $l < r$

$s \leftarrow \text{Partition}(A[l..r])$

$\text{Quicksort}(A[l..s-1])$

$\text{Quicksort}(A[s+1..r])$

ALGORITHM Partition($A[l..r]$)

//Partitions a subarray by Hoare's algorithm, using the first element as a pivot

//Input: Subarray of array $A[0..n - 1]$, defined by its left and right indices l and r ($l < r$)

//Output: Partition of $A[l..r]$, with the split position returned as this function's value

$p \leftarrow A[l]$

$i \leftarrow l; j \leftarrow r + 1$

repeat

repeat $i \leftarrow i + 1$ until $A[i] \geq p$

repeat $j \leftarrow j - 1$ until $A[j] \leq p$

$\text{swap}(A[i], A[j])$

until $i \geq j$

$\text{swap}(A[i], A[j])$ //undo last swap when $i \geq j$

$\text{swap}(A[l], A[j])$

return j

PROGRAM

```
#include<stdio.h>

#include<conio.h>

#include<stdlib.h>

#include<time.h>

int partition(int a[],int low,int high)
{
    int i,j,temp,key;
    key=a[low];
    i=low+1;
    j=high;
    while(1)
    {
        while(i<high && key>=a[i])
            i++;
        while(key<a[j])
            j--;
        if(i<j)
        {
            temp=a[i];
            a[i]=a[j];
            a[j]=temp;
        }
        else
        {
            temp=a[low];
            a[low]=a[j];
            a[j]=temp;
        }
        return j;
    }
}
```

```

}

void quicksort(int a[ ],int low,int high)
{
    int j;
    if(low<high)
    {
        j=partition (a, low, high);
        quicksort (a,low,j-1);
        quicksort(a,j+1,high);
    }
}

int main()
{
    int i ,n,a[100];
    time_t start,end;
    printf("enter the number of elements\n");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        a[i]=rand();
        printf("%d\t",a[i]) ;
    }
    start=clock();
    quicksort(a,0,n-1);
    end=clock();
    printf("\nthe sorted array is \n");
    for(i=0;i<n;i++)
    {
        printf("%d\t",a[i]);
    }
    double tc=(difftime(end,start)/CLOCKS_PER_SEC);
    printf("\ntime taken is %f",tc);
    return 0;
}

```

```
}
```

Output:

enter the number of elements

10

41 18467 6334 26500 19169 15724 11478 29358 26962 24464

the sorted array is

41 6334 11478 15724 18467 19169 26500 24464 26962 29358

time taken is 0.000000

2. **Develop the logic using Merge sort strategy and implement the program to sort a given set of elements and determine the time required to sort the elements. Repeat the experiment for different values of n , the number of elements in the list to be sorted and plot a graph for the time taken versus n . The elements should be read from a file/can be generated using the random number generator.**

ALGORITHM Mergesort($A[0..n - 1]$)

//Sorts array $A[0..n - 1]$ by recursive mergesort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in non-decreasing order

if $n > 1$ copy $A[0.. n/2 - 1]$ to $B[0.. n/2 - 1]$

copy $A[n/2.. n - 1]$ to $C[0.. n/2 - 1]$

Mergesort($B[0.. n/2 - 1]$)

Mergesort($C[0.. n/2 - 1]$)

Merge(B, C, A)

ALGORITHM Merge($B[0..p - 1], C[0..q - 1], A[0..p + q - 1]$)

//Merges two sorted arrays into one sorted array

//Input: Arrays $B[0..p - 1]$ and $C[0..q - 1]$ both sorted

//Output: Sorted array $A[0..p + q - 1]$ of the elements of B and C

$i \leftarrow 0; j \leftarrow 0; k \leftarrow 0$

while $i < p$ and $j < q$

if $B[i] \leq C[j]$

$A[k] \leftarrow B[i]; i \leftarrow i + 1$

else $A[k] \leftarrow C[j]; j \leftarrow j + 1$

$k \leftarrow k + 1$

```

if i = p
    copy C[j..q - 1] to A[k..p + q - 1]
else copy B[i..p - 1] to A[k..p + q - 1]

```

PROGRAM

```

#include<stdio.h>
#include<stdlib.h>
#include<dos.h>
#include<time.h>
void merge(int arr[], int l, int m, int r)
{
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        r[j] = arr[m + 1 + j];
    i = 0;
    j = 0;
    k = l;
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
    }
}

```



```

        }
        k++;
    }
    while (i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }
    while (j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergesort(int arr[], int l, int r)
{
    if (l < r)
    {
        int m = l+(r-l)/2;
        mergesort(arr, l, m);
        mergesort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}

int main()
{
    int i,n,a[100];
    time_t start,end;
    printf("enter the number of elements\n");
    scanf("%d",&n);
    for(i=0;i<n;i++)

```

```

{
    a[i]=rand();
    printf("%d\t",a[i]) ;
}
start=clock();
mergesort(a,0,n-1);
end=clock();
printf("\nsorted array elements are\n");
for(i=0;i<n;i++)
{
    printf("%d\t",a[i]);
}
double tc=(difftime(end,start)/CLOCKS_PER_SEC);
printf("\ntime taken is %f",tc);
return 0;
}

```

Output:

enter the number of elements

10

41 18467 6334 26500 19169 15724 11478 29358 26962 24464

sorted array elements are

41 6334 11478 15724 18467 19169 24464 26500 26962 29358

time taken is 0.000000

3. Write and execute a menu driven program to print all the nodes reachable from a given starting node in a graph using DFS and BFS method.

ALGORITHM dfs(G)

//Implements a depth-first search traversal of a given graph

//Input: Graph $G = V, E$

//Output: Graph G with its vertices marked with consecutive integers in the order they are first encountered by the DFS traversal mark each vertex in V with 0 as a mark of being “unvisited”

count $\leftarrow$ 0

for each vertex v in V do

 if v is marked with 0

 dfs(v)

dfs(v)

//visits recursively all the unvisited vertices connected to vertex v by a path and numbers them in the order they

//are encountered via global variable count

count $\leftarrow$ count + 1;

mark v with count

for each vertex w in V adjacent to v do

 if w is marked with 0

 dfs(w)

ALGORITHM bfs(G)

//Implements a breadth-first search traversal of a given graph

//Input: Graph $G = V, E$

//Output: Graph G with its vertices marked with consecutive integers

// in the order they are visited by the bfs traversal mark each vertex in V with 0 as a mark of being “unvisited”

count $\leftarrow$ 0

for each vertex v in V do

 if v is marked with 0

 bfs(v)

bfs(v)

count $\leftarrow$ count + 1; mark v with count and initialize a queue with v

while the queue is not empty do

```
    for each vertex w in V adjacent to the front vertex do
        if w is marked with 0
            count  $\leftarrow$  count + 1; mark w with count
            add w to the queue
    remove the front vertex from the queue
```

PROGRAM

```
#include <stdio.h>

#include <stdlib.h>

#define MAX_NODES 100

// Define a structure for a node in the graph
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// Define a structure for the graph
typedef struct Graph {
    int numNodes;
    Node* adjLists[MAX_NODES];
} Graph;

// Function to create a new node
Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to add an edge to the graph
```

```

void addEdge(Graph* graph, int src, int dest) {
    Node* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;
}

// Function to perform Depth First Search (DFS)
void DFS(Graph* graph, int vertex, int visited[]) {
    visited[vertex] = 1;
    printf("%d ", vertex);

    Node* temp = graph->adjLists[vertex];
    while (temp != NULL) {
        int adjVertex = temp->data;
        if (!visited[adjVertex])
            DFS(graph, adjVertex, visited);
        temp = temp->next;
    }
}

// Function to perform Breadth First Search (BFS)
void BFS(Graph* graph, int start) {
    int visited[MAX_NODES] = {0};
    int queue[MAX_NODES];
    int front = -1, rear = -1;

    visited[start] = 1;
    queue[++rear] = start;

    while (front != rear) {
        int currentVertex = queue[++front];
        printf("%d ", currentVertex);
    }
}

```

```

Node* temp = graph->adjLists[currentVertex];
while (temp != NULL) {
    int adjVertex = temp->data;
    if (!visited[adjVertex]) {
        visited[adjVertex] = 1;
        queue[++rear] = adjVertex;
    }
    temp = temp->next;
}
}
}

```

```

int main() {
    int choice, startNode;
    Graph* graph = (Graph*)malloc(sizeof(Graph));
    graph->numNodes = 0;

    do {
        printf("\nMenu:\n");
        printf("1. Add Edge\n");
        printf("2. Print reachable nodes using DFS\n");
        printf("3. Print reachable nodes using BFS\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: {
                int src, dest;

                printf("Enter source and destination nodes: ");
                scanf("%d %d", &src, &dest);

                if (src >= MAX_NODES || dest >= MAX_NODES || src < 0 || dest < 0) {
                    printf("Invalid input!\n");
                }
            }
        }
    } while (choice != 4);
}

```

```

        break;
    }
    if (src > graph->numNodes) graph->numNodes = src + 1;
    if (dest > graph->numNodes) graph->numNodes = dest + 1;
    addEdge(graph, src, dest);
    break;
}
case 2: {
    printf("Enter the starting node for DFS: ");
    scanf("%d", &startNode);
    printf("Nodes reachable from %d using DFS: ", startNode);
    int visitedDFS[MAX_NODES] = {0};
    DFS(graph, startNode, visitedDFS);
    printf("\n");
    break;
}
case 3: {
    printf("Enter the starting node for BFS: ");
    scanf("%d", &startNode);
    printf("Nodes reachable from %d using BFS: ", startNode);
    BFS(graph, startNode);
    printf("\n");
    break;
}
case 4: {
    printf("Exiting...\n");
    break;
}
default:
    printf("Invalid choice!\n");
}
} while (choice != 4);

```

```

// Free allocated memory
for (int i = 0; i < MAX_NODES; i++) {
    Node* current = graph->adjLists[i];
    while (current != NULL) {
        Node* temp = current;
        current = current->next;
        free(temp);
    }
}
free(graph);

return 0;
}

```

Output:

Menu:

1. Add Edge
2. Print reachable nodes using DFS
3. Print reachable nodes using BFS
4. Exit

Enter your choice: 1

Enter source and destination nodes: 0 1

Menu:

1. Add Edge
2. Print reachable nodes using DFS
3. Print reachable nodes using BFS
4. Exit

Enter your choice: 1

Enter source and destination nodes: 0 2

Menu:

1. Add Edge
2. Print reachable nodes using DFS
3. Print reachable nodes using BFS
4. Exit

Enter your choice: 2

Enter the starting node for DFS: 0

Nodes reachable from 0 using DFS: 0 2 1

Menu:

1. Add Edge
2. Print reachable nodes using DFS
3. Print reachable nodes using BFS
4. Exit

Enter your choice: 3

Enter the starting node for BFS: 0

Nodes reachable from 0 using BFS: 0 1 2

Menu:

1. Add Edge
2. Print reachable nodes using DFS
3. Print reachable nodes using BFS
4. Exit

Enter your choice: 4

Exiting...

4. Write and execute a menu driven program to print all the nodes reachable from a given starting node in a graph to arrange nodes in topological order using DFS method and source removal technique

ALGORITHM : Topsort(G)

//Input : G is a directed acyclic graph (DAG)

T $\leftarrow$ empty list

Z $\leftarrow$ empty stack .

in $\leftarrow$ dictionary mapping all vertices to 0

for each $v \in V$ do

 for each u adjacent to v do

 increment in[v]

for each $v \in V$ do

 if in[v] = 0 then

 add v to Z

while S is not empty do

 v $\leftarrow$ Z.remove

 append v to T

 for each u adjacent to v do

 decrement in[u]

 if in[u] = 0 then

 add u to Z

return T

ALGORITHM: Steps involved in finding the topological ordering of a DAG:

Step-1: Compute in-degree (number of incoming edges) for each of the vertex present in the DAG and initialize the count of visited nodes as 0.

Step-2: Pick all the vertices with in-degree as 0 and add them into a queue (Enqueue operation)

Step-3: Remove a vertex from the queue (Dequeue operation) and then.

 Increment the count of visited nodes by 1.

 Decrease in-degree by 1 for all its neighboring nodes.

 If the in-degree of neighboring nodes is reduced to zero, then add it to the queue.

Step 4: Repeat Step 3 until the queue is empty.

Step 5: If the count of visited nodes is not equal to the number of nodes in the graph then the topological sort is not possible for the given graph.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>

int s[100], count, res[100];

//-----
void dfs(int n, int a[100][100], int source)
{
    int i;
    s[source] = 1;
    for (i = 1; i <= n; i++) {
        if (a[source][i] == 1 && s[i] == 0)
            dfs(n, a, i);
    }
    count = count + 1;
    res[count] = source;
}
//-----
void topologicalDFS(int n, int a[100][100]) {
    int i, j;
    for (i = 1; i <= n; i++)
    {
        s[i] = 0;
    }
    count = 0;
    for (j = 1; j <= n; j++)
    {
        if (s[j] == 0)
        {
            dfs(n, a, j);
        }
    }
    for (i = n; i >= 1; i--)
    {
        printf("--> %d ", res[i]);
    }
    printf("\n");
}
```

```

void topologicalSourceRemoval(int n, int
a[100][100], int sc[100]) { int i, j;
for (i = 1; i <= n; i++) {
    if (sc[i] == 0) {
        printf("%d\t", i);
        sc[i] = -1;
        for (j = 1; j <= n; j++) {
            if (a[i][j] == 1) {
                sc[j] = sc[j] - 1;
            }
        }
        i = 0;
    }
}
printf("\n");
}
//-----
----- int main()
{
    int a[100][100], n, i, j, sc[100],ch;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++)
            scanf("%d", &a[i][j]);

    printf("1:Topological order using DFS 2:
Source Removal method:\n");
    scanf("%d",&ch);

    switch(ch)
    {
    case 1: printf("Topological order using DFS:\n");
            topologicalDFS(n, a);
            break;

    case 2: for (i = 1; i <= n; i++)
            sc[i] = 0;

            for (i = 1; i <= n; i++)
                for (j = 1; j <= n; j++)
                    if(a[i][j] == 1)
                    {
                        sc[j] = sc[j] + 1;
                    }
            printf("Topological order using Source Removal:\n");

```

```

        topologicalSourceRemoval(n, a, sc);
        break;

    default: exit(0);
}
}

```

```

/*
Enter number of vertices: 7
Enter adjacency matrix:
0 1 1 0 0 0 0
0 0 0 0 1 0 1
0 0 0 0 0 1 0
1 1 1 0 0 1 1
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 1 1 0
1:Topological order using DFS 2:
Source Removalmethod: 1
Topological order using DFS:
--> 4 --> 1 --> 3 --> 2 --> 7 --> 6 --> 5
Enter number of vertices: 7
Enter adjacency matrix:
0 1 1 0 0 0 0
0 0 0 0 1 0 1
0 0 0 0 0 1 0
1 1 1 0 0 1 1
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 1 1 0
1:Topological order using DFS 2: Source Removalmethod:
2
Topological order using Source Removal:
4 1 2 3 7 5 6

```

5. Write and execute a program to search for the given pattern string in given text string using Horspool String Matching algorithm.

ALGORITHM ShiftTable(P[0..m – 1])

//Fills the shift table used by Horspool's and Boyer-Moore algorithms

//Input: Pattern P[0..m – 1] and an alphabet of possible characters

//Output: Table[0..size – 1] indexed by the alphabet's characters and

// filled with shift sizes

for i ← 0 to size – 1 do

Table[i] ← m

```

for j ← 0 to m - 2 do
    Table[P[j]] ← m - 1 - j
return Table

```

ALGORITHM HorspoolMatching(P[0..m - 1], T [0..n - 1])

```

//Implements Horspool's algorithm for string matching
//Input: Pattern P[0..m - 1] and text T [0..n - 1] /
//Output: The index of the left end of the first matching substring or -1 if there are no matches
ShiftTable(P[0..m - 1]) //generate Table of shifts
i ← m - 1 //position of the pattern's right end
while i ≤ n - 1 do
    k ← 0 //number of matched characters
    while k ≤ m - 1 and P[m - 1 - k] = T [i - k] do
        k ← k + 1
    if k = m
        return i - m + 1
    else i ← i + Table[T [i]]
return -1

```

PROGRAM

```

#include <stdio.h>
#include <string.h>
#define MAX_TEXT_LENGTH 100
#define MAX_PATTERN_LENGTH 10
#define ALPHABET_SIZE 128

int horspool(char text[], int textLength, char pattern[], int patternLength)
{
    int shiftTable[ALPHABET_SIZE];
    int i, j;
    for (i = 0; i < ALPHABET_SIZE; i++)
    {
        shiftTable[i] = patternLength;
    }
}

```

```

for (i = 0; i < patternLength - 1; i++)
{
    shiftTable[int(pattern[i])] = patternLength - 1 - i;
}
i = patternLength - 1;
while (i < textLength)
{
    j = 0;
    while (j < patternLength && text[i - j] == pattern[patternLength - 1 - j])
    {
        j++;
    }
    if (j == patternLength)
    {
        return i - patternLength + 1;
    }
    i += shiftTable[int(text[i])];
}
return -1; // Pattern not found
}

int main()
{
    char text[MAX_TEXT_LENGTH];
    char pattern[MAX_PATTERN_LENGTH];
    int textLength, patternLength;
    int matchIndex;
    printf("Enter the text: ");
    fgets(text, sizeof(text), stdin);
    textLength = strlen(text);
    if (text[textLength - 1] == '\n')
    {
        text[textLength - 1] = '\0';
        textLength--;
    }
}

```

```

printf("Enter the pattern: ");
fgets(pattern, sizeof(pattern), stdin);
patternLength = strlen(pattern);
if (pattern[patternLength - 1] == '\n')
{
    pattern[patternLength - 1] = '\0';
    patternLength--;
}
matchIndex = horspool(text, textLength, pattern, patternLength);
if (matchIndex != -1)
{
    printf("Pattern found at index %d\n", matchIndex);
}
else
{
    printf("Pattern not found\n");
}
return 0;
}

```

OUTPUT1:

Enter the text: analysis and design of algorithms

Enter the pattern: design

Pattern found at index 13

OUTPUT2:

Enter the text: analysis and design of algorithms

Enter the pattern: compiler

Pattern not found

6. Write and execute a program for matrix chain multiplication for the given the sequence of Matrices $\langle A_1, A_2, \dots, A_n \rangle$.

ALGORITHM: MATRIX-CHAIN-MULTIPLICATION(p)

```

n = p.length - 1
let m[1...n, 1...n] and s[1...n - 1, 2...n] be new matrices
for i = 1 to n
    m[i, i] = 0
for l = 2 to n // l is the chain length
    for i = 1 to n - l + 1
        j = i + l - 1
        m[i, j] =  $\infty$ 
        for k = i to j - 1
            q = m[i, k] + m[k + 1, j] + pi-1 * pk * pj
            if q < m[i, j]
                m[i, j] = q
                s[i, j] = k
return m and s

```

PROGRAM

```

#include <limits.h>

#include <stdio.h>

int MatrixChainOrder(int p[], int i, int j)
{
    if (i == j)
        return 0;

    int k;

    int min = INT_MAX;

    int count;

    for (k = i; k < j; k++)
    {
        count = MatrixChainOrder(p, i, k) + MatrixChainOrder(p, k + 1, j)
                + p[i - 1] * p[k] * p[j];

        if (count < min)
            min = count;
    }

    return min;
}

```

```
int main()
{
    int arr[] = { 1, 2, 3, 4, 3 };
    int N = sizeof(arr) / sizeof(arr[0]);
    printf("Minimum number of multiplications is %d ",MatrixChainOrder(arr, 1, N - 1));
    return 0;
}
```

OUTPUT:

Minimum number of multiplications is 30

7. Write and execute a program to find a longest-common-subsequence of X and Y using dynamic programming for given two sequences $X = \langle x_1, x_2, \dots, x_m \rangle$ $Y = \langle y_1, y_2, \dots, y_n \rangle$

ALGORITHM:

X and Y be two given sequences

Initialize a table LCS of dimension $X.length * Y.length$

X.label = X

Y.label = Y

LCS[0][] = 0

LCS[][0] = 0

Start from LCS[1][1]

Compare X[i] and Y[j]

 if $X[i] = Y[j]$

$LCS[i][j] = 1 + LCS[i-1, j-1]$

 Point an arrow to LCS[i][j]

 else

$LCS[i][j] = \max(LCS[i-1][j], LCS[i][j-1])$

 Point an arrow to $\max(LCS[i-1][j], LCS[i][j-1])$

PROGRAM

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int i, j, m, n, LCS_table[20][20];
```

```
char S1[20] = "ACADB", S2[20] = "CBDA", b[20][20];
```

```
void lcsAlgo()
```

```
{
```

```
    m = strlen(S1);
```

```
    n = strlen(S2);
```

```
    for (i = 0; i <= m; i++)
```

```
        LCS_table[i][0] = 0;
```

```
    for (i = 0; i <= n; i++)
```

```
        LCS_table[0][i] = 0;
```

```

for (i = 1; i <= m; i++)
    for (j = 1; j <= n; j++)
    {
        if (S1[i - 1] == S2[j - 1])
        {
            LCS_table[i][j] = LCS_table[i - 1][j - 1] + 1;
        }
        else if (LCS_table[i - 1][j] >= LCS_table[i][j - 1])
        {
            LCS_table[i][j] = LCS_table[i - 1][j];
        }
        else
        {
            LCS_table[i][j] = LCS_table[i][j - 1];
        }
    }

    int index = LCS_table[m][n];
char lcsAlgo[index + 1];
lcsAlgo[index] = '\0';
int i = m, j = n;
while (i > 0 && j > 0)
{
    if (S1[i - 1] == S2[j - 1])
    {
        lcsAlgo[index - 1] = S1[i - 1];
        i--;
        j--;
        index--;
    }
    else if (LCS_table[i - 1][j] > LCS_table[i][j - 1])
        i--;

```

```

        else
            j--;
    }
    printf("S1 : %s \nS2 : %s \n", S1, S2);
    printf("LCS: %s", lcsAlgo);
}

int main()
{
    lcsAlgo();
    printf("\n");
}

```

OUTPUT:

S1 : ACADB

S2 : CBDA

LCS: CB

8. Write and execute a program to find Minimum Spanning Tree using Prim's method

ALGORITHM Prim(G)

//Prim's algorithm for constructing a minimum spanning tree

//Input: A weighted connected graph $G = V, E$

//Output: E_T , the set of edges composing a minimum spanning tree of G

$V_T \leftarrow \{v_0\}$ //the set of tree vertices can be initialized with any vertex

$E_T \leftarrow \emptyset$

for $i \leftarrow 1$ to $|V| - 1$ do

 find a minimum-weight edge $e^* = (v^*, u^*)$ among all the edges (v, u) such that v is in V_T and u is in $V - V_T$

$V_T \leftarrow V_T \cup \{u^*\}$

$E_T \leftarrow E_T \cup \{e^*\}$

return E_T

PROGRAM

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#include <limits.h>
```

```
#define MAX_VERTICES 100
```

```
int graph[MAX_VERTICES][MAX_VERTICES];
```

```
int numVertices;
```

```
int findMinimumKey(int key[], bool mstSet[])
```

```
{
```

```
    int minKey = INT_MAX;
```

```
    int minIndex, v;
```

```
    for (v = 0; v < numVertices; v++)
```

```
    {
```

```
        if (mstSet[v] == false && key[v] < minKey)
```

```
        {
```

```
            minKey = key[v];
```

```
            minIndex = v;
```

```
        }
```

```
    }
```

```

        return minIndex;
    }

void printMST(int parent[])
{
    int i;
    printf("Edge \tWeight\n");
    for (i = 1; i < numVertices; i++)
    {
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
    }
}

void primMST()
{
    int parent[MAX_VERTICES];
    int key[MAX_VERTICES];
    bool mstSet[MAX_VERTICES];
    int i, count, u, v;
    for (i = 0; i < numVertices; i++)
    {
        key[i] = INT_MAX;
        mstSet[i] = false;
    }
    key[0] = 0;
    parent[0] = -1;
    for (count = 0; count < numVertices - 1; count++)
    {
        u = findMinimumKey(key, mstSet);
        mstSet[u] = true;
        for (v = 0; v < numVertices; v++)
        {
            if (graph[u][v] && mstSet[v] == false && graph[u][v] < key[v])
            {
                parent[v] = u;
            }
        }
    }
}

```

```

        key[v] = graph[u][v];
    }
}

}

printMST(parent);
}

int main()
{
    int i, j;
    printf("Enter the number of vertices: ");
    scanf("%d", &numVertices);
    printf("Enter the adjacency matrix:\n");
    for (i = 0; i < numVertices; i++)
    {
        for (j = 0; j < numVertices; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }
    printf("Minimum Spanning Tree:\n");
    primMST();
    return 0;
}

```

OUTPUT:

Enter the number of vertices: 6

Enter the adjacency matrix:

0 4 4 999 999 999

4 0 2 999 999 999

4 2 0 3 2 4

999 999 3 0 999 3

999 999 2 999 0 3

999 999 4 3 3 0

Minimum Spanning Tree:

Edge	Weight
------	--------

0 - 1	4
-------	---

1 - 2	2
-------	---

2 - 3	3
-------	---

2 - 4	2
-------	---

4 - 5	3
-------	---

9. Write and execute a program to find Minimum Spanning Tree using Kruskal's method

ALGORITHM Kruskal(G)

//Kruskal's algorithm for constructing a minimum spanning tree

//Input: A weighted connected graph $G = V, E$

//Output: E_T , the set of edges composing a minimum spanning tree of G sort E in non-decreasing order of the edge weights $w(e_{i1}) \leq \dots \leq w(e_{i|E|})$

$E_T \leftarrow \emptyset$; ecounter $\leftarrow 0$ //initialize the set of tree edges and its size

$k \leftarrow 0$ //initialize the number of processed edges

while ecounter $< |V| - 1$ do

$k \leftarrow k + 1$

 if $E_T \cup \{e_{ik}\}$ is acyclic

$E_T \leftarrow E_T \cup \{e_{ik}\}$; ecounter $\leftarrow$ ecounter + 1

return E_T

PROGRAM

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int comparator(const void* p1, const void* p2)
```

```
{
```

```
    const int(*x)[3] = p1;
```

```
    const int(*y)[3] = p2;
```

```
    return (*x)[2] - (*y)[2];
```

```
}
```

```
void makeSet(int parent[], int rank[], int n)
```

```
{
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        parent[i] = i;
```

```
        rank[i] = 0;
```

```
    }
```

```
}
```

```
int findParent(int parent[], int component)
```

```
{
```

```

        if (parent[component] == component)
            return component;

        return parent[component] = findParent(parent, parent[component]);
    }

void unionSet(int u, int v, int parent[], int rank[], int n)
{
    u = findParent(parent, u);
    v = findParent(parent, v);
    if (rank[u] < rank[v])
    {
        parent[u] = v;
    }
    else if (rank[u] > rank[v])
    {
        parent[v] = u;
    }
    else
    {
        parent[v] = u;
        rank[u]++;
    }
}

void kruskalAlgo(int n, int edge[n][3])
{
    qsort(edge, n, sizeof(edge[0]), comparator);
    int parent[n];
    int rank[n];
    makeSet(parent, rank, n);
    int minCost = 0;
    printf("Following are the edges in the constructed MST\n");
    for (int i = 0; i < n; i++)
    {
        int v1 = findParent(parent, edge[i][0]);

```

```

        int v2 = findParent(parent, edge[i][1]);
        int wt = edge[i][2];
        if (v1 != v2)
        {
            unionSet(v1, v2, parent, rank, n);
            minCost += wt;
            printf("%d -- %d = %d\n", edge[i][0], edge[i][1], wt);
        }
    }

    printf("Minimum Cost Spanning Tree: %d\n", minCost);
}

int main()
{
    int edge[5][3] = { { 0, 1, 10 },
                        { 0, 2, 6 },
                        { 0, 3, 5 },
                        { 1, 3, 15 },
                        { 2, 3, 4 } };

    kruskalAlgo(5, edge);

    return 0;
}

```

OUTPUT:

Following are the edges in the constructed MST

2 -- 3 = 4

0 -- 3 = 5

0 -- 1 = 10

Minimum Cost Spanning Tree: 19

10. Write and execute a program to find shortest path to all other nodes in weighted graph using Dijkstra's strategy

ALGORITHM Dijkstra(G, s)

//Dijkstra's algorithm for single-source shortest paths

//Input: A weighted connected graph $G = V, E$ with nonnegative weights and its vertex s

//Output: The length d_v of a shortest path from s to v and its penultimate vertex p_v for every vertex v in V

Initialize(Q) //initialize priority queue to empty

for every vertex v in V

$d_v \leftarrow \infty$; $p_v \leftarrow \text{null}$

 Insert(Q, v , d_v) //initialize vertex priority in the priority queue

$d_s \leftarrow 0$; Decrease(Q, s , d_s) //update priority of s with d_s

$VT \leftarrow \emptyset$

for $i \leftarrow 0$ to $|V| - 1$ do

$u^* \leftarrow \text{DeleteMin}(Q)$ //delete the minimum priority element

$VT \leftarrow VT \cup \{u^*\}$

 for every vertex u in $V - VT$ that is adjacent to u^* do

 if $d_{u^*} + w(u^*, u) < d_u$

$d_u \leftarrow d_{u^*} + w(u^*, u)$; $p_u \leftarrow u^*$

 Decrease(Q, u , d_u)

PROGRAM

```
#include <limits.h>
```

```
#include <stdbool.h>
```

```
#include <stdio.h>
```

```
#define V 9
```

```
int minDistance(int dist[], bool sptSet[])
```

```
{
```

```
    int min = INT_MAX, min_index;
```

```
    for (int v = 0; v < V; v++)
```

```
        if (sptSet[v] == false && dist[v] <= min)
```

```
            min = dist[v], min_index = v;
```

```
    return min_index;
```

```
}
```

```

void printSolution(int dist[])
{
    printf("Vertex \t\t Distance from Source\n");
    for (int i = 0; i < V; i++)
        printf("%d \t\t\t %d\n", i, dist[i]);
}

void dijkstra(int graph[V][V], int src)
{
    int dist[V];
    bool sptSet[V];
    for (int i = 0; i < V; i++)
        dist[i] = INT_MAX, sptSet[i] = false;
    dist[src] = 0;
    for (int count = 0; count < V - 1; count++)
    {
        int u = minDistance(dist, sptSet);
        sptSet[u] = true;
        for (int v = 0; v < V; v++)
            if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX && dist[u] +
                graph[u][v] < dist[v])
                dist[v] = dist[u] + graph[u][v];
    }
    printSolution(dist);
}

int main()
{
    int graph[V][V] = { { 0, 4, 0, 0, 0, 0, 0, 8, 0 },
                        { 4, 0, 8, 0, 0, 0, 0, 11, 0 },
                        { 0, 8, 0, 7, 0, 4, 0, 0, 2 },
                        { 0, 0, 7, 0, 9, 14, 0, 0, 0 },
                        { 0, 0, 0, 9, 0, 10, 0, 0, 0 },
                        { 0, 0, 4, 14, 10, 0, 2, 0, 0 },

```

```

        { 0, 0, 0, 0, 0, 2, 0, 1, 6 },
        { 8, 11, 0, 0, 0, 0, 1, 0, 7 },
        { 0, 0, 2, 0, 0, 0, 6, 7, 0 }

};

dijkstra(graph, 0);

return 0;

}

```

OUTPUT:

Vertex	Distance from Source
0	0
1	4
2	12
3	19
4	21
5	11
6	9
7	8
8	14

11. Write and execute a program to find a subset of a given set $S = \{s_1, s_2, \dots, s_n\}$ of n positive integers whose sum is equal to a given positive integer d . For example, if $S = \{1, 2, 5, 6, 8\}$ and $d = 9$ there are two solutions $\{1, 2, 6\}$ and $\{1, 8\}$. A suitable message is to be displayed if the given problem instance doesn't have a solution

ALGORITHM SUB_SET_PROBLEM, (i, sum, W, remSum)

// Description: Solve sub of subset problem using backtracking //Input: W: Number for which subset is to be computed i: Item index sum: Sum of integers selected so far remSum: Size of remaining problem i.e. (W - sum)

// Output: Solution tuple X

if FEASIBLE SUB_SET() == 1 then

 if (sum == W) then

 print X [1...i]

 end

else

 X [i + 1] <- 1

 SUB SET PROBLEM(i + 1, sum + w[i] + 1, W, remSum - w * [i] + 1)

 X[i+1] <- 0 //Exclude the " item

 SUB_SET_PROBLEM(i + 1, sum, W, remSum- w[i] + 1)

end

FEASIBLE_SUB_SET()

 if (sum+remSum ≥ W) AND (sum == W) or (sum + w[i] + 1 ≤ W) then

 return 0

 end

 return 1

PROGRAM

```
#include <stdio.h>
```

```
const int max=10;
```

```
int s[10], x[10];
```

```
int d,flag=0;
```

```
void sumofsub(int,int,int);
```

```
int main()
```

```
{
```

```
    int i,n,sum=0;
```

```
    printf("Enter maximum no\n");
```

```
    scanf("%d",&n);
```

```
    printf("Enter the set in increasing order\n");
```



```

    for(int i=1;i<=n;i++)
        scanf("%d",&s[i]);
    printf("Enter the maximum subset value:");
    scanf("%d",&d);
    for(i=1;i<=n;i++)
        sum+=s[i];
    if(sum<d || s[1]>d)
    {
        printf("No subset possible\n");
        flag=1;
    }
    sumofsub(0,1,sum);
    if(flag==0)
        printf("no subset possible\n");
    return 0;
}

void sumofsub(int m,int k,int r)
{
    x[k]=1;
    if((m+s[k])==d)
    {
        printf("{}");
        for(int i=1;i<=k;i++)

            if(x[i]==1)
            {
                printf(" %d ",s[i]);
                flag=1;
            }
        printf("{}");
        printf("\n");
    }
    else
    {
        if(m+s[k]+s[k+1]<=d)
            sumofsub(m+s[k],k+1,r-s[k]);
        if((m+r-s[k]>=d) && (m+s[k+1]<=d))

```

```

        {
            x[k]=0;
            sumofsub(m,k+1,r-s[k]);
        }
    }
}

```

OUTPUT:

Enter maximum no

5

Enter the set in increasing order

1

2

3

4

5

Enter the maximum subset value:5

{ 1 4 }

{ 2 3 }

{ 5 }

12. Write and execute a program to find solution to n- queens problem

ALGORITHM

```
function solveNQueens(board, col, n):
    if col >= n:
        print board
        return true
    for row from 0 to n-1:
        if isSafe(board, row, col, n):
            board[row][col] = 1
            if solveNQueens(board, col+1, n):
                return true
            board[row][col] = 0
    return false
```

```
function isSafe(board, row, col, n):
    for i from 0 to col-1:
        if board[row][i] == 1:
            return false
    for i,j from row-1, col-1 to 0, 0 by -1:
        if board[i][j] == 1:
            return false
    for i,j from row+1, col-1 to n-1, 0 by 1, -1:
        if board[i][j] == 1:
            return false
    return true
```

board = empty NxN chessboard

solveNQueens(board, 0, N)

ALGORITHM Backtrack(X[1..i])

//Gives a template of a generic backtracking algorithm

//Input: X[1..i] specifies first i promising components of a solution

//Output: All the tuples representing the problem's solutions

if X[1..i] is a solution

 write X[1..i]

else

 for each element $x \in S_{i+1}$ consistent with X[1..i] and the constraints do

$X[i + 1] \leftarrow x$

Backtrack(X[1..i + 1])

PROGRAM

```
#include<stdio.h>
#include<math.h>
int a[30],count=0;
int place(int pos)
{
    int i;
    for (i=1;i<pos;i++)
    {
        if(((a[i]==a[pos])||((abs(a[i]-a[pos])==abs(i-pos))))))
            return 0;
    }
    return 1;
}
void print_sol(int n)
{
    int i,j;
    count++;
    printf("\n\nSolution # %d:\n",count);
    for (i=1;i<=n;i++)
    {
        for (j=1;j<=n;j++)
        {
            if(a[i]==j)
                printf("Q\t"); else
                printf("*\t");
        }
        printf("\n");
    }
}
```

```

}
void queen(int n)
{
    int k=1;
    a[k]=0;
    while(k!=0)
    {
        a[k]=a[k]+1;
        while((a[k]<=n)&&!place(k))
            a[k]++;
        if(a[k]<=n)
        {
            if(k==n)
                print_sol(n);
            else
            {
                k++;
                a[k]=0;
            }
        }
        else
            k--;
    }
}

int main()
{
    int n;
    printf("Enter the number of Queens\n");
    scanf("%d",&n);
    queen(n);
    printf("\nTotal solutions=%d",count);
}

```

```
        return 0;  
    }
```

OUTPUT:

Enter the number of Queens

4

Solution #1:

```
*   Q   *   *  
*   *   *   Q  
Q   *   *   *  
*   *   Q   *
```

Solution #2:

```
*   *   Q   *  
Q   *   *   *  
*   *   *   Q  
*   Q   *   *
```

Total solutions=2

VIVA QUESTIONS

1. What is an algorithm?

An algorithm is a sequence of unambiguous instructions for solving a problem. i.e., for obtaining a required output for any legitimate input in a finite amount of time

2. What are important problem types?

1. Sorting 2. Searching 3. Numerical problems 4. Geometric problems 5. Combinatorial Problems 6. Graph Problems 7. String processing Problems

3. Name some basic Efficiency classes

1. Constant 2. Logarithmic 3. Linear 4. $n \log n$ 5. Quadratic 6. Cubic 7. Exponential 8. Factorial

4. What are algorithm design techniques?

Algorithm design techniques (or strategies or paradigms) are general approaches to solving problems algorithmically, applicable to a variety of problems from different areas of computing. General design techniques are:

(i) Brute force (ii) divide and conquer (iii) decrease and conquer (iv) transform and conquer (v) greedy technique (vi) dynamic programming (vii) backtracking (viii) branch and bound

5. How is an algorithm's time efficiency measured?

Time efficiency indicates how fast the algorithm runs. An algorithm's time efficiency is measured as a function of its input size by counting the number of times its basic operation (running time) is executed. Basic operation is the most time consuming operation in the algorithm's innermost loop.

6. How is the efficiency of the algorithm defined?

The efficiency of an algorithm is defined with the components. (i) Time efficiency -indicates how fast the algorithm runs (ii) Space efficiency -indicates how much extra memory the algorithm needs

7. What are the characteristics of an algorithm?

Every algorithm should have the following five characteristics (i) Input (ii) Output (iii) Definiteness (iv) Effectiveness (v) Termination. Therefore, an algorithm can be defined as a sequence of definite and effective instructions, which terminates with the production of correct output from the given input.

8. Write general plan for analyzing non-recursive algorithms.

i. Decide on parameter indicating an input's size. ii. Identify the algorithm's basic operation iii. Checking the no. of times basic operation executed depends on size of input. iv. Set up sum expressing the no. of times the basic operation is executed. Depends on some additional property, then best, worst, avg cases need to be investigated (establishing order of growth)

9. Define the terms: pseudocode, flow chart

A pseudocode is a mixture of a natural language and programming language like constructs. A pseudocode is usually more precise than natural language. A flowchart is a method of expressing an algorithm by a collection of connected geometric shapes containing descriptions of the algorithm's steps.

10. Write general plan for analyzing recursive algorithms.

i. Decide on parameter indicating an input's size. ii. Identify the algorithm's basic operation iii. Checking the no. of times basic operation executed depends on size of input. if it depends on some additional property, then best, worst, avg cases need to be investigated of times the basic operation is executed. iv. Set up the recurrence relation, with an appropriate initial condition, for the number v. Solve recurrence (establishing order of growth)

11. Define the divide and conquer method.

Given a function to compute on 'n' inputs the divide-and-conquer strategy suggests splitting the inputs in to 'k' distinct subsets, $1 < k < n$, yielding 'k' sub problems. The sub problems must be solved recursively, and then a method must be found to combine sub solutions into a solution of the whole.

12. What is Merge sort?

Merge sort is divide and conquer strategy that works by dividing an input array in to two halves, sorting them recursively and then merging the two sorted halves to get the original array sorted.

13. What is general divide and conquer recurrence?

Time efficiency $T(n)$ of many divide and conquer algorithms satisfies the equation $T(n) = aT(n/b) + f(n)$. This is the general recurrence relation.

14. Explain the greedy method.

Greedy method is the most important design technique, which makes a choice that looks best at that moment. A given 'n' inputs are required us to obtain a subset that satisfies some constraints that is the feasible solution. A greedy method suggests that one can device an algorithm that works in stages considering one input at a time.

15. Define feasible and optimal solution.

Given n inputs and we are required to form a subset such that it satisfies some given constraints then such a subset is called feasible solution. A feasible solution either maximizes or minimizes the given objective function is called as optimal solution

16. What are the constraints of knapsack problem?

To maximize $\sum p_i x_i$ The constraint is : $\sum w_i x_i \geq m$ and $0 \leq x_i \leq 1$ $1 \leq i \leq n$

Where m is the bag capacity, n is the number of objects and for each object i w_i and p_i are the weight and profit of object respectively.

17. Specify the algorithms used for constructing Minimum cost spanning tree.

- a) Prim's Algorithm
- b) Kruskal's Algorithm

18. State single source shortest path algorithm (Dijkstra's algorithm).

For a given vertex called the source in a weighted connected graph, find shortest paths to all its other vertices. Dijkstra's algorithm applies to graph with non-negative weights only.

19. State efficiency of prim's algorithm.

$O(|V|^2)$ (weight matrix and priority queue as unordered array)

$O(|E| \log |V|)$ (adjacency list and priority queue as min-heap)

20. State Kruskal Algorithm.

The algorithm looks at a MST for a weighted connected graph as an acyclic sub graph with $|V|-1$ edges for which the sum of edge weights is the smallest.

21. State efficiency of Dijkstra's algorithm.

$O(|V|^2)$ (weight matrix and priority queue as unordered array)

$O(|e| \log |v|)$ (adjacency list and priority queue as min-heap)

22. Define multistage graph

A multistage graph $G = (V, E)$ is a directed graph in which the vertices are partitioned into $K \geq 2$ disjoint sets $V_i, 1 \leq i \leq k$. The multi stage graph problem is to find a minimum cost paths from s (source) to t (sink) Two approach (forward and backward)

23. Define all pair shortest path problem.

Given a weighted connected graph, all pair shortest path problem asks to find the lengths of shortest paths from each vertex to all other vertices.

24. Define Floyd's algorithm

To find all pair shortest path

25. State the time efficiency of Floyd's algorithm

$O(n^3)$ It is cubic

26. What is the time complexity of linear search?

$\Theta(n)$

27. What is the time complexity of binary search?

$\Theta(\log_2 n)$

28. What is the major requirement for binary search?

The given list should be sorted.

29. What is binary search?

It is an efficient method of finding out a required item from a given list, provided the list is in order. The process is: 1. First the middle item of the sorted list is found. 2. Compare the item with this element. 3. If they are equal search is complete. 4. If the middle element is greater than the item being searched, this process is repeated in the upper half of the list. 5. If the middle element is lesser than the item being searched, this process is repeated in the lower half of the list.

30. What is parental dominance?

The key at each node is greater than or equal to the keys at its children.

31. What is heap?

A **heap** can be defined as a binary tree with keys assigned to its nodes (one key per node) provided the following two conditions are met: 1 The tree's shape requirement – The binary tree is essentially complete (or simply complete), that is, all its levels are full except possibly the last level, where only some rightmost leaves may be missing. 2. The parental dominance requirement – The key at each node is greater than or equal to the keys at its children

32. What is height of Binary tree?

It is the longest path from the root to any leaf.

33. What is the time complexity of heap sort?

$\Theta(n \log n)$

34. What is Merge Sort?

Merge sort is an $O(n \log n)$ comparison-based sorting algorithm. Where the given array is divided into two equal parts. The left part of the array as well as the right part of the array is sorted recursively. Later, both the left sorted part and right sorted part are merged into a single sorted array.

35. Who invented Merge Sort?

John Von Neumann in 1945.

36. On which paradigm is it based?

Merge-sort is based on the divide-and-conquer paradigm.

37. What is the time complexity of merge sort?

$n \log n$.

38. What is the space requirement of merge sort?

Space requirement: $\Theta(n)$ (not in-place).

39. When do you say an algorithm is stable and in place?

Stable – if it retains the relative ordering.

In– place if it does not use extra memory.

40. Who invented Dijkstra’s Algorithm?

Edsger Dijkstra invented this algorithm.

41. What is the other name for Dijkstra’s Algorithm?

Single Source Shortest Path Algorithm.

42. Which technique the Dijkstra’s algorithm is based on?

Greedy Technique.

43. What is the purpose of Dijkstra’s Algorithm?

To find the shortest path from source vertex to all other remaining vertices

44. Name the different algorithms based on Greedy technique.

Prim’s Algorithm, Kruskal’s algorithm

45. What is the constraint on Dijkstra’s Algorithm?

This algorithm is applicable to graphs with non-negative weights only.

46. What is a Weighted Graph?

A weighted graph is a graph with numbers assigned to its edges. These numbers are called weights or costs.

47. What is a Connected Graph?

A graph is said to be connected if for every pair of its vertices u and v there is a path from u to v .

48. What is the time complexity of Dijkstra’s algorithm?

For adjacency matrix, It is $O(V^2)$ For adjacency list with edges stored as min heap, it is $O(E \log V)$

49. Differentiate b/w Traveling Salesman Problem (TSP) and Dijkstra’s Algorithm.

In TSP, given n cities with known distances b/w each pair, find the shortest tour that passes through

all the cities exactly once before returning to the starting city. In Dijkstra's Algorithm, find the shortest path from source vertex to all other remaining vertices

50. Differentiate b/w Prim's Algorithm and Dijkstra's Algorithm?

Prim's algorithm is to find minimum cost spanning tree. Dijkstra's algorithm is to find the shortest path from source vertex to all other remaining vertices.

51. What are the applications of Dijkstra's Algorithm?

It finds its application even in our day to day life while travelling. They are helpful in routing applications.

52. Who was the inventor of the Quicksort?

C.A.R.HOARE, a British Scientist.

53. Which design strategy does Quicksort uses?

Divide and Conquer

54. What is Divide and Conquer Technique?

(I) Divide the instance of a problem into two or more smaller instances (II) Solve the smaller instances recursively (III) Obtain solution to original instances by combining these solutions

55. What is another name for Quicksort?

Partition Exchange Sort

56. Is Quicksort Stable as well as In place?

Not Stable but In place.

57. When do you say that a Quick sort having best case complexity?

When the pivot exactly divides the array into equal half

58. When do you say that Quick sort having worst case complexity?

When any one of the sub array is empty

59. How many more comparisons are needed for average case compared to best case?

38% more

60. When do you say that the pivot element is in its final position?

When all the elements towards the left of pivot are $\leq$ pivot and all the elements towards the right of pivot are $\geq$ pivot.

61. What technique does kruskal's algorithm follow?

Greedy technique

62. What is the time complexity of Kruskal algorithm.

With an efficient sorting algorithm, the time efficiency of an kruskal's algorithm will be in $O(|E|\log|E|)$.

63. Who is the inventor of kruskal's algorithm?

Joseph Kruskal.

64. Which technique is used to solve BFS & DFS problems?

Decrease and Conquer

65. What is DFS traversal?

Consider an arbitrary vertex v and mark it as visited on each iteration, proceed to an unvisited vertex w adjacent to v . we can explore this new vertex depending upon its adjacent information. This process continues until dead end is encountered.(no adjacent unvisited vertex is available). At the dead end the algorithm backs up by one edge and continues the process of visiting unvisited vertices. This process continues until we reach the starting vertex.

66. What are the various applications of BFS & DFS tree traversal technique?

Application of BFS

Checking connectivity and checking acyclicity of a graph.

Check whether there is only one root in the BFS forest or not.

If there is only one root in the BFS forest, then it is connected graph otherwise disconnected graph.

For checking cycle presence, we can take advantage of the graph's representation in the form of a BFS forest, if the latter vertex does not have cross edge, then the graph is acyclic.

Application of DFS

To check whether given graph is connected or not. To check whether the graph is cyclic or not. To find the spanning tree.

Topological sorting.

67. Which data structures are used in BFS & DFS tree traversal technique?

We use Stack data structure to traverse the graph in DFS traversal.

We use queue data structure to traverse the graph in BFS traversal.

68. What is Dynamic Programming?

It is a technique for solving problems with overlapping sub problems.

69. What does Dynamic Programming have in common with Divide and Conquer?

Both solve the problems by dividing the problem into sub problems. Using the solutions of sub problems, the solutions for larger instance of the problem can be obtained.

70. What is a principle difference between the two techniques?

Only one instance of the sub problem is computed & stored. If the same instance of sub problem is encountered, the solution is retrieved from the table and never recomputed. Very precisely re-computations are not preformed.

71. What is a spanning tree?

A spanning tree of a weighted connected graph is a connected acyclic (no cycles) sub graph (i.e. a tree) that contains all the vertices of a graph and number of edges is one less than number of vertices.

72. What is a minimum spanning tree?

Minimum spanning tree of a connected graph is its spanning tree of smallest weight.

73. Prim's algorithm is based on which technique.

Greedy technique

74. Name some of the application greedy technique

They are helpful in routing applications: In the case of network where large number of computers are connected, to pass the data from one node to another node we can find the shortest distance easily by this algorithm. Communication Networks: Suppose there are 5 different cities and we want to

establish computer connectivity among them, then we take into the account of cost factor for communication links. Building a road network that joins n cities with minimum cost.

75. Does Prim's Algorithm always yield a minimum spanning tree ?

Yes

76. What is the purpose of Floyd's algorithm?

To find the all pair shortest path.

77. What is the time efficiency of Floyd's algorithm?

It is same as warshall's algorithm that is $\theta(n^3)$.

78. What is shortest path?

It is the path which is having shortest length among all possible paths.

79. Warshall's algorithm is optimization problem or non-optimization problem ?

Non-optimization problem

80. For what purpose we use Warshall's algorithm?

Warshall's algorithm for computing the transitive closure of a directed graph

81. Warshall's algorithm depends on which property?

Transitive property

82. What is the time complexity of Warshall's algorithm?

Time complexity of warshall's algorithm is $\theta(n^3)$.

83. What is state space tree?

Constructing a tree of choices being made and processing is known as state-spacetree. Root represents an initial state before the search for solution begins.

84. What are promising and non-promising state-space-tree?

Node which may leads to solution is called promising. Node which doesn't leads to solution is called

non-promising.

85. What are the requirements that are needed for performing Backtracking?

Complex set of constraints. They are: i. Explicit constraints. ii. Implicit constraints.